

IMDb Movie Rating Scraper – Post Project Report

Automated Movie Data Extraction and Analysis

Author: R M Gayathri

Project Type: Individual Project

EXECUTIVE SUMMARY:

The **IMDb Movie Rating Scraper** project demonstrates the implementation of an automated Python-based system designed to extract movie data from the IMDb website. The tool automates the collection of the **Top 250 movies**, including their **title, release year, and IMDb rating**, and stores them in a clean, structured **CSV file** for further analysis. The project leverages Selenium WebDriver with Chrome in **headless mode**, ensuring fully automated and efficient data extraction without manual intervention.

PROJECT OVERVIEW:

Objective:

To develop a dynamic web scraper that automatically extracts IMDb's Top 250 movie data using browser automation, overcoming manual data collection limitations and enabling real-time retrieval even from dynamically loaded pages.

Scope:

1. Web scraping from IMDb's Top 250 Movies chart.
2. Extraction of movie details: Title, Release Year, and IMDb Rating.
3. Removal of unwanted serial numbers from titles for clean output.

4. Export of structured movie data into CSV format for easy reuse.
5. Fully automated and compatible with headless execution mode.

Key Features:

1. **Dynamic Data Fetching:** Retrieves the latest Top 250 movies automatically whenever the script runs.
 2. **Accurate Extraction:** Handles IMDb's modern React-based layout and extracts data using dynamic selectors.
 3. **Clean Data Output:** Removes unnecessary serial numbers and saves only relevant movie details.
 4. **Headless Execution:** Runs in background mode without showing any browser window.
 5. **Structured CSV Export:** Generates a clean `imdb_top_250.csv` file ready for data analysis or visualization.
-

TECHNICAL IMPLEMENTATION:

Architecture:

1. The system follows a straightforward scraping pipeline:
Selenium Browser Automation → Access IMDb Top 250 webpage.
2. Data Extraction → Scrape movie title, year, and rating using CSS selectors.
3. Data Cleaning → Remove serial numbers and unwanted symbols using regex.
4. Export Generation → Store the cleaned data in CSV format.

Technology Stack:

1. Python 3.x – Core programming language.
2. Selenium – For automated web browsing and dynamic scraping.

3. pandas – For tabular data manipulation and export to CSV.
4. re (Regular Expressions) – For removing serial numbers in titles.
5. time – To handle load delays during page rendering.

Performance Metrics:

1. Data Volume: Top 250 IMDb Movies per execution.
2. Execution Time: 8–10 seconds per run.
3. Output Format: Clean CSV file (imdb_top_250.csv).

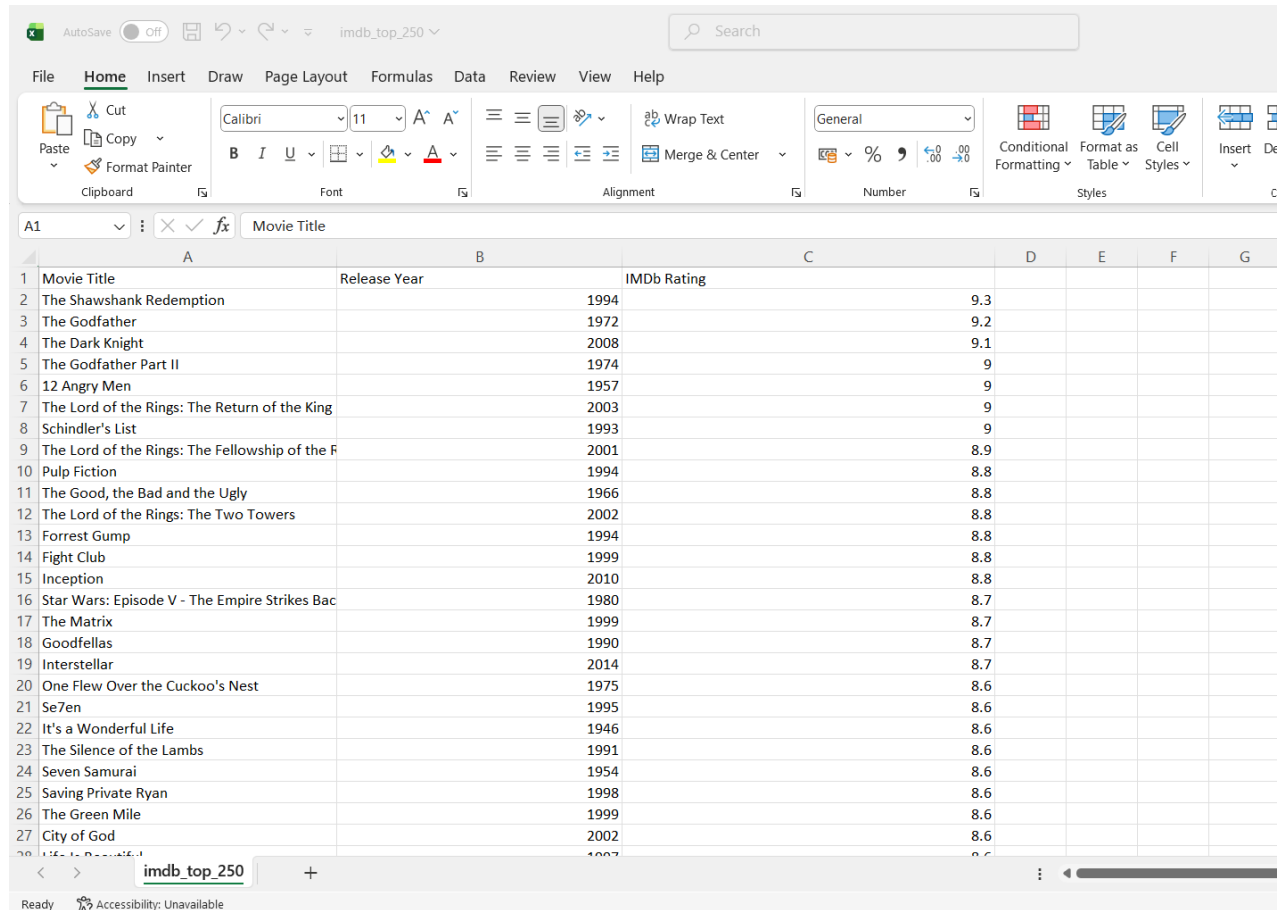
SOURCE CODE IMPLEMENTATION:

```
imdb_scraper.py •
imdb_scraper.py > ...
1  from selenium import webdriver
2  from selenium.webdriver.chrome.service import Service
3  from selenium.webdriver.chrome.options import Options
4  from selenium.webdriver.common.by import By
5  import pandas as pd
6  import time
7  import re
8
9  driver_path = r"C:\Users\gayat\Downloads\chromedriver-win64\chromedriver-win64\c
10
11  chrome_options = Options()
12  chrome_options.add_argument("--headless")
13  chrome_options.add_argument("--no-sandbox")
14  chrome_options.add_argument("--disable-dev-shm-usage")
15  chrome_options.add_argument("--window-size=1920,1080")
16  chrome_options.add_argument(
17      "user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
18      "(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
19  )
20
21  # Start driver
22  service = Service(driver_path)
23  driver = webdriver.Chrome(service=service, options=chrome_options)
24
```

```
imdb_scraper.py •
imdb_scraper.py > ...
23 driver = webdriver.Chrome(service=service, options=chrome_options)
24
25 print("📡 Loading IMDb Top 250 page (Headless Mode)...")
26 driver.get("https://www.imdb.com/chart/top")
27
28 time.sleep(8)
29
30 movies = driver.find_elements(By.CSS_SELECTOR, "li.ipc-metadata-list-summary-item")
31
32 titles, years, ratings = [], [], []
33
34 for movie in movies:
35     try:
36         title_raw = movie.find_element(By.CSS_SELECTOR, "h3.ipc-title__text").text
37         title = re.sub(r"^\d+\.\s*", "", title_raw)
38
39         year = movie.find_element(By.CSS_SELECTOR, "span.cli-title-metadata-item")
40         rating = movie.find_element(By.CSS_SELECTOR, "span.ipc-rating-star--rating")
41
42         titles.append(title)
43         years.append(year)
44         ratings.append(rating)
45     except Exception:
46         continue
```

```
File Edit Selection View ... Mini Project 2
imdb_scraper.py •
imdb_scraper.py > ...
41
42     titles.append(title)
43     years.append(year)
44     ratings.append(rating)
45 except Exception:
46     continue
47
48 # Create CSV
49 df = pd.DataFrame({
50     "Movie Title": titles,
51     "Release Year": years,
52     "IMDb Rating": ratings
53 })
54
55 df.to_csv("imdb_top_250.csv", index=False)
56
57 print(f"✅ Successfully scraped {len(df)} movies in headless mode!")
58 print("📁 Clean file saved as 'imdb_top_250.csv'")
59
60 driver.quit()
61
```

EXTRACTED MOVIE DATA:



AutoSave Off | imdb_top_250 | Search

File Home Insert Draw Page Layout Formulas Data Review View Help

Clipboard: Paste, Cut, Copy, Format Painter

Font: Calibri, 11, Bold, Italic, Underline, Text Color, Background Color

Alignment: Wrap Text, Merge & Center

Number: General, Percentage, Currency, Accounting, Decimals

Styles: Conditional Formatting, Format as Table, Cell Styles

Insert: Rows, Columns, Tables, Charts, Links, Comments, Hyperlinks, PivotTables, PivotCharts, Slicers, Sparklines

	A	B	C	D	E	F	G
	Movie Title	Release Year	IMDb Rating				
1	The Shawshank Redemption	1994	9.3				
2	The Godfather	1972	9.2				
3	The Dark Knight	2008	9.1				
4	The Godfather Part II	1974	9				
5	12 Angry Men	1957	9				
6	The Lord of the Rings: The Return of the King	2003	9				
7	Schindler's List	1993	9				
8	The Lord of the Rings: The Fellowship of the Ring	2001	8.9				
9	Pulp Fiction	1994	8.8				
10	The Good, the Bad and the Ugly	1966	8.8				
11	The Lord of the Rings: The Two Towers	2002	8.8				
12	Forrest Gump	1994	8.8				
13	Fight Club	1999	8.8				
14	Inception	2010	8.8				
15	Star Wars: Episode V - The Empire Strikes Back	1980	8.7				
16	The Matrix	1999	8.7				
17	Goodfellas	1990	8.7				
18	Interstellar	2014	8.7				
19	One Flew Over the Cuckoo's Nest	1975	8.6				
20	Se7en	1995	8.6				
21	It's a Wonderful Life	1946	8.6				
22	The Silence of the Lambs	1991	8.6				
23	Seven Samurai	1954	8.6				
24	Saving Private Ryan	1998	8.6				
25	The Green Mile	1999	8.6				
26	City of God	2002	8.6				
27	Life is Beautiful	1997	8.6				

Ready | Accessibility: Unavailable

KEY ACHIEVEMENTS:

Technical Accomplishments:

1. Successfully automated IMDb data extraction using Selenium in headless mode.
2. Implemented modern CSS selectors compatible with IMDb's new React structure.
3. Integrated regular expressions to clean serial numbers from movie titles.
4. Generated accurate, structured CSV output for further analysis.

Process Automation Benefits:

1. Consistency: Automated script ensures reliable data extraction each run.
 2. Efficiency: Reduces manual effort and speeds up data collection.
 3. Reusability: CSV data can be reused for data visualization or machine learning.
-

LIMITATIONS AND CONSTRAINTS:

Current Limitations:

- Website Dependency: The scraper depends on IMDb's HTML layout. Any future layout changes may require selector updates.
- Internet Requirement: Live internet connection is necessary for scraping.
- Local Execution: Requires ChromeDriver installed locally for Selenium automation.

Resource Constraints:

- No cloud data storage or API integration in the current version.
 - Limited to the Top 250 Movies chart; does not scrape genre, cast, or description yet.
-

PROJECT IMPACT AND VALUE:

Immediate Benefits:

- Eliminates manual IMDb browsing for movie data.
- Produces accurate and organized information for quick reference.
- Saves time and reduces repetitive data collection work.

Long-term Potential:

- Enables movie trend analysis or visualization dashboards.
 - Can be expanded to collect cast, genre, or language data.
 - Provides a foundation for developing a personalized movie recommendation system.
-

CONCLUSION:

The **IMDb Movie Rating Scraper** project successfully demonstrates how Selenium-based automation can be used for data collection from dynamic websites like IMDb. The scraper efficiently retrieves the Top 250 movie details, cleans and structures them, and exports the results into a CSV file. This automation improves accuracy, consistency, and speed in collecting entertainment data. Future enhancements could include genre extraction, sentiment-based ranking, and integration with AI-powered recommendation systems — transforming it into a complete **movie data analytics platform**.